

Warehouse Network Carrier Performance Analytics Platform

1. Project Overview

Project Title

Warehouse Network Carrier Performance Analytics Platform

Objective

The objective of this project is to build an end-to-end cloud-based analytics platform that processes warehouse and carrier shipment data, performs ETL operations, stores processed data in a data warehouse, and generates analytical insights for business decision-making.

Business Problem

Logistics companies generate huge amounts of shipment and carrier data. Analyzing this data manually is difficult and time-consuming. The project aims to automate data ingestion, transformation, storage, and reporting.

Expected Outcomes

- Automated data ingestion
- Data transformation using PySpark
- Data storage in AWS
- Analytical reporting
- Visualization dashboards

2. Dataset Details

Dataset Name

Supply Chain Logistics Dataset

Source

https://brunel.figshare.com/articles/dataset/Supply_Chain_Logistics_Problem_Dataset/7558679

Provider: Figshare (Brunel University London)

Dataset Overview

This dataset represents a real-world warehouse network and logistics system used to analyze order routing, carrier selection, warehouse capacity, and transportation costs.

It models how orders are distributed across warehouses, shipped through different carriers, and delivered to destination ports under real operational constraints.

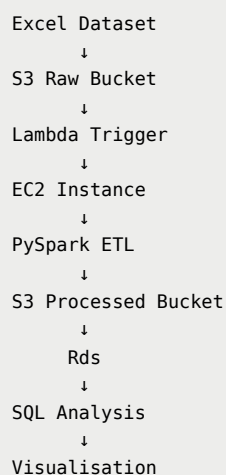
The dataset is ideal for building end-to-end data engineering pipelines, as it includes multiple related tables that require joining, transformation, and analytical processing.

3. Architecture Diagram

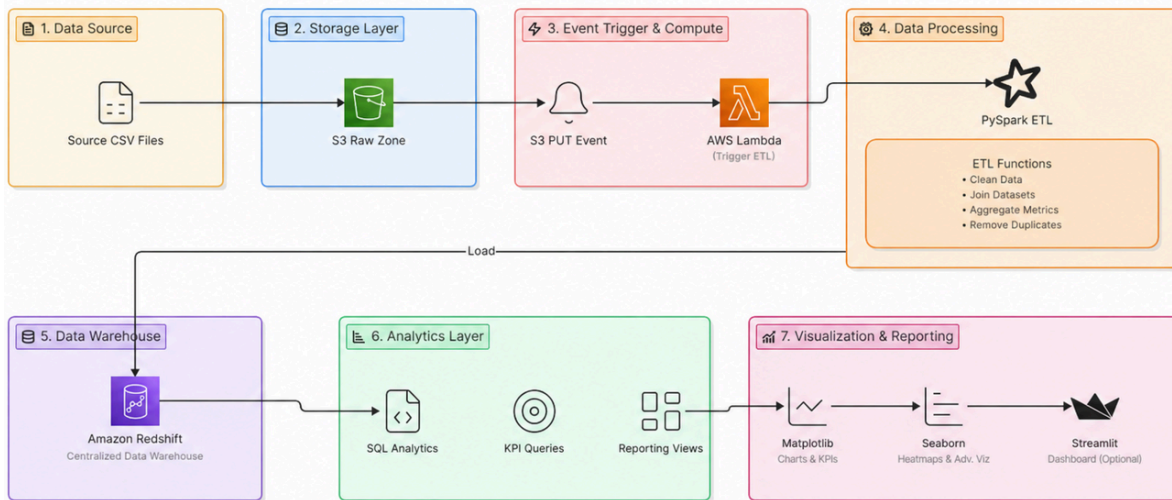
The Warehouse Network Carrier Performance Analytics Platform is designed to analyze shipment, carrier, and warehouse operational data using a cloud-based data engineering pipeline on AWS. The solution automates data ingestion, processing, storage, and analytics to provide actionable insights into logistics performance, delivery efficiency, and transportation costs.

The architecture leverages AWS services such as Amazon S3, AWS Lambda, Amazon EC2, PySpark, and Amazon Redshift to build a scalable and efficient analytics platform.

Example Flow:



Project Architecture



4. Cloud Resources

The Warehouse Network Carrier Performance Analytics Platform leverages multiple AWS cloud services to build a scalable, reliable, and automated data engineering pipeline. Each service plays a specific role in the data ingestion, processing, storage, and analytics workflow.

Amazon S3 (Simple Storage Service)

Amazon S3 serves as the primary storage layer for the project.

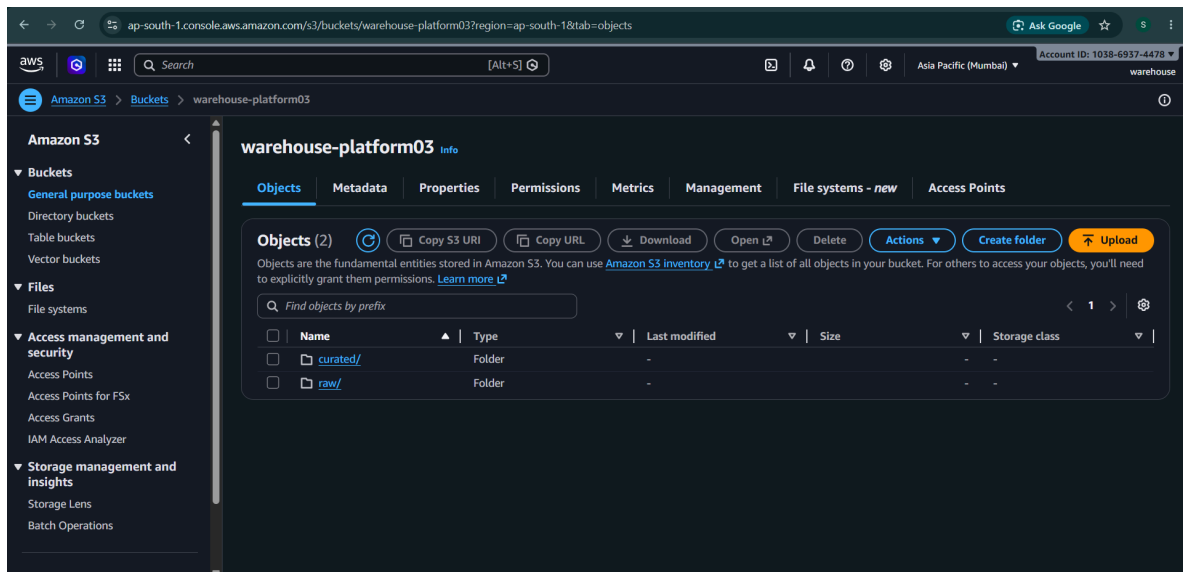
Two separate buckets are maintained:

S3 Raw Bucket

- Stores source datasets in their original format.
- Acts as the landing zone for incoming data.
- Preserves raw data for auditing and reprocessing purposes.

S3 Processed Bucket

- Stores cleaned and transformed datasets generated by the ETL process.
- Serves as the trusted data layer for downstream analytics.



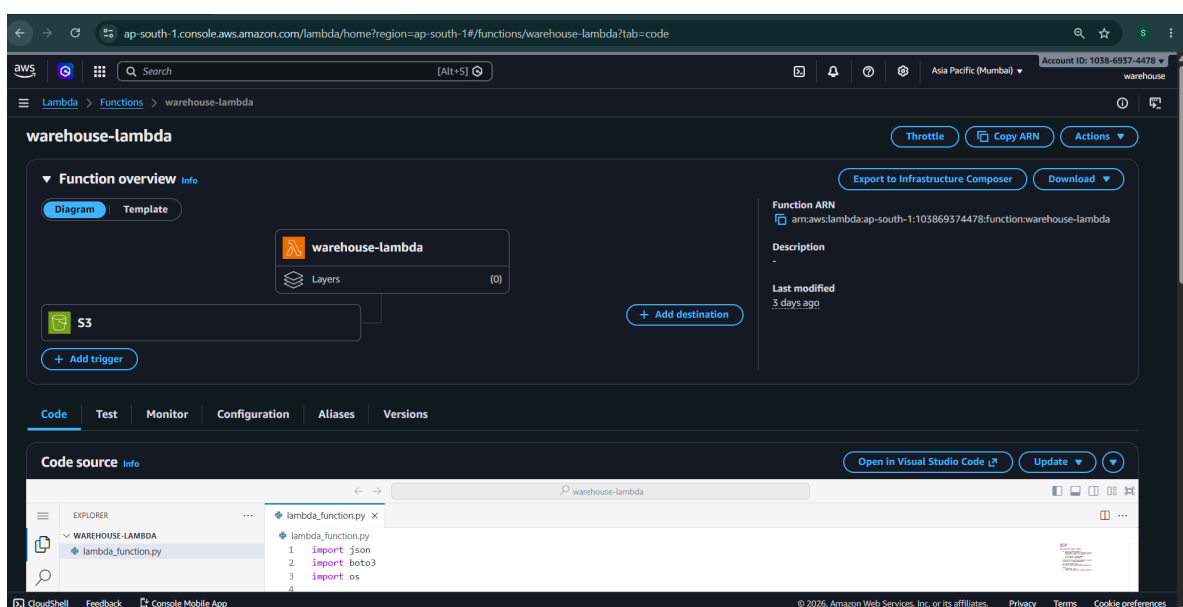
AWS Lambda

AWS Lambda is used as the event-driven orchestration service.

Whenever a new file is uploaded to the Raw S3 Bucket, an S3 event notification automatically triggers a Lambda function.

Responsibilities

- Detect new file uploads.
- Trigger the ETL workflow.
- Start the EC2 instance used for PySpark processing.
- Automate pipeline execution without manual intervention.



Amazon EC2 (Elastic Compute Cloud)

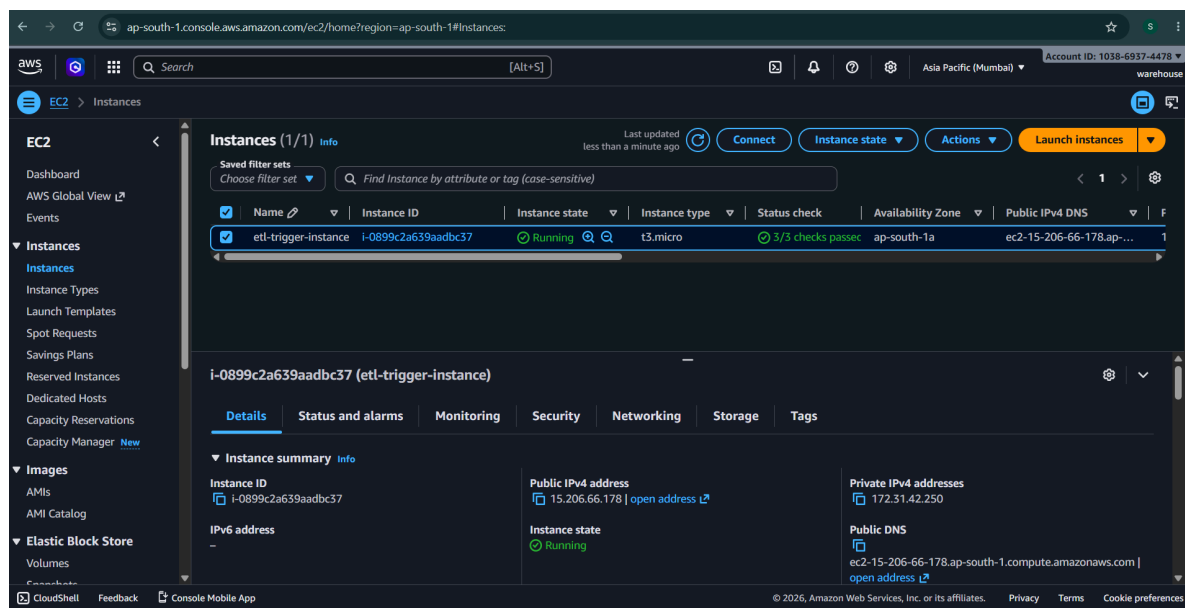
Amazon EC2 provides the compute environment required for ETL processing.

The EC2 instance hosts:

- Python environment
- PySpark
- Required libraries and dependencies

Responsibilities

- Execute PySpark ETL jobs.
- Read raw files from Amazon S3.
- Perform data cleansing and transformations.
- Write processed data back to Amazon S3.

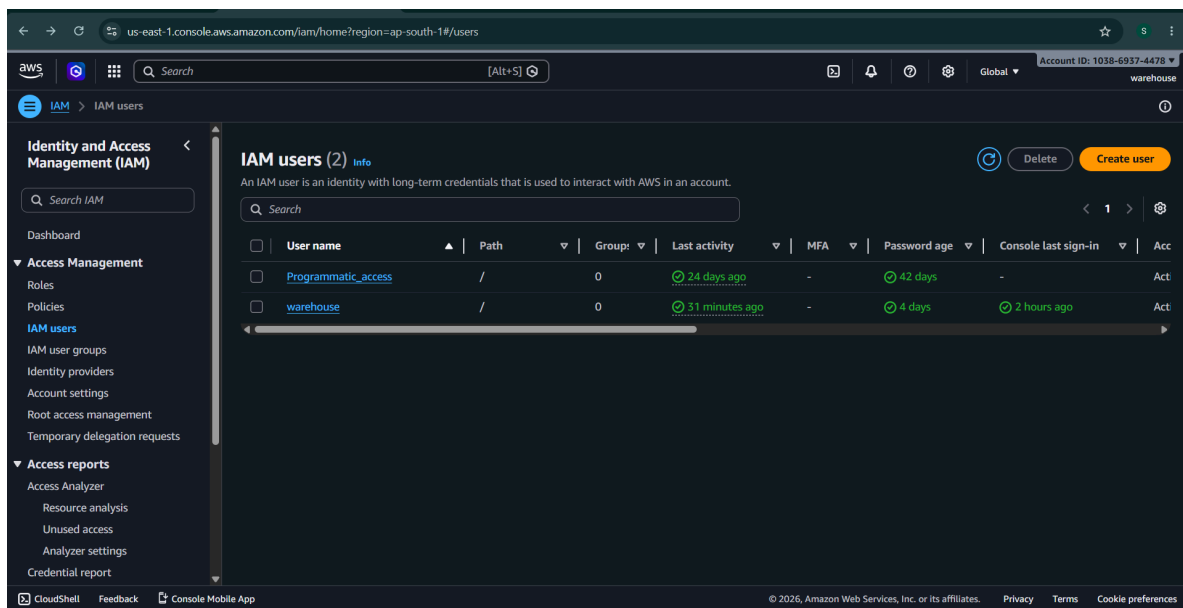


AWS IAM (Identity and Access Management)

AWS IAM is used to securely manage permissions and access across AWS services.

Responsibilities

- Control access to S3 buckets.
- Allow Lambda to interact with EC2.
- Enable EC2 to access S3 resources.
- Secure RDS (PostgreSQL) connectivity.



Amazon RDS (PostgreSQL)

Amazon RDS (Relational Database Service) serves as the centralized relational database for the Warehouse Network Carrier Performance Analytics Platform.

Processed datasets from Amazon S3 are loaded into AWS RDS (PostgreSQL) for structured storage, SQL-based analytics, and reporting.

Responsibilities

- Store structured warehouse, carrier, and shipment performance data.
- Support relational schema design (fact and dimension tables).
- Execute SQL-based analytical queries for business insights.
- Maintain data consistency, integrity, and relational constraints.

5. Database Design

The Warehouse Network Carrier Performance Analytics Platform uses **AWS RDS (PostgreSQL)** as the final relational database for storing processed and transformed data. After ETL processing using PySpark, raw datasets are cleaned, joined, and structured into analytical tables before being loaded into the database.

The database is designed to support efficient SQL-based analytics for warehouse operations, carrier performance, and logistics optimization.

Database Overview

The database is structured to support the following business analysis areas:

- Warehouse capacity utilization

- Carrier transit performance
- Order routing and delivery analysis
- Cost and efficiency tracking
- Logistics risk scoring

The design follows a **relational star-schema inspired model**, optimized for analytical queries.

Core Tables (Post ETL)

1. fact_orders (Main Transaction Table)

This is the central fact table created after joining and transforming all raw datasets.

Columns:

- order_id (Primary Key)
- product_id (Foreign Key)
- plant_id (Foreign Key)
- carrier_id (Foreign Key)
- origin_port
- destination_port
- unit_quantity
- unit_weight
- order_date (derived during ETL)
- estimated_transit_days
- estimated_delivery_date

Purpose:

Stores all shipment-level transactional data used for analytics.

2. dim_warehouses

Stores warehouse master information.

Columns:

- plant_id (Primary Key)
- daily_capacity
- unit_storage_cost

Purpose:

Used for calculating warehouse utilization and cost efficiency.

3. dim_carriers

Stores carrier and route performance data.

Columns:

- carrier_id (Primary Key)
- origin_port
- destination_port
- tpt_day_cnt (transit days)

Purpose:

Used for analyzing carrier speed and performance.

4. dim_products

Stores product master data.

Columns:

- product_id (Primary Key)

Purpose:

Used for product-level analytics and mapping.

5. bridge_products_per_plant

Handles many-to-many relationship between warehouses and products.

Columns:

- plant_id (Foreign Key)
- product_id (Foreign Key)

Purpose:

Defines which products can be stored in which warehouses.

Analytics Tables (Derived Tables)

These tables are generated during the ETL transformation phase for reporting purposes.

6. analytics.warehouse_health

Tracks warehouse utilization and cost efficiency.

Columns:

- plant_id
 - report_date
 - daily_order_count
 - utilization_pct
 - unit_storage_cost
-

7. analytics.carrier_performance

Analyzes carrier speed and efficiency.

Columns:

- carrier_id
 - destination_port
 - avg_transit_days
 - order_count
 - rank_by_speed
-

8. analytics.order_routing_priority

Provides risk-based ranking of orders.

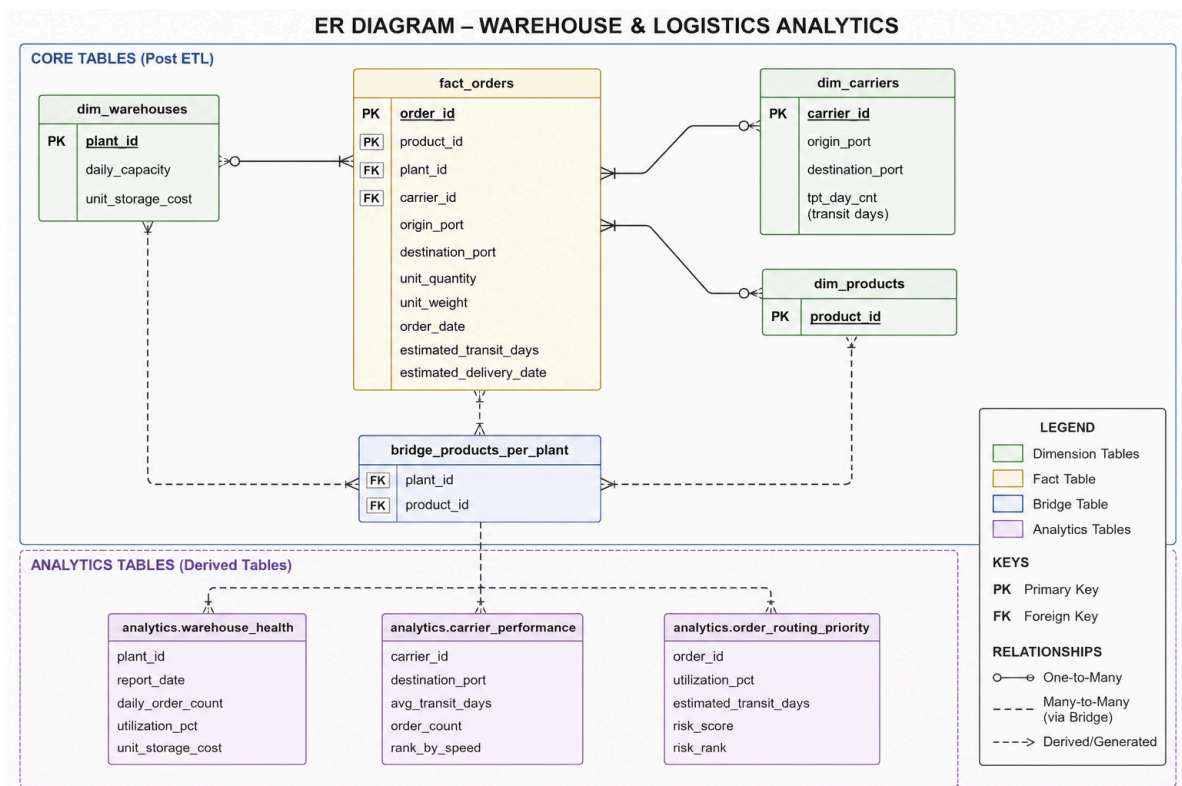
Columns:

- order_id
 - utilization_pct
 - estimated_transit_days
 - risk_score
 - risk_rank
-

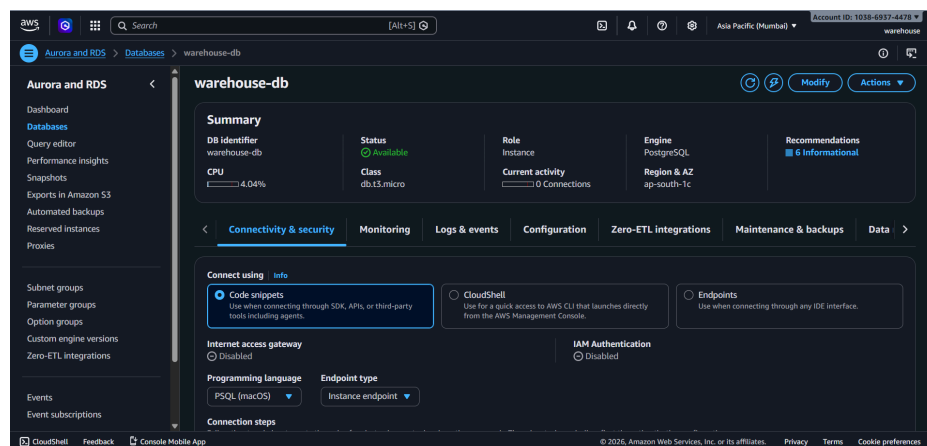
Data Relationships

- fact_orders → dim_warehouses (plant_id)
 - fact_orders → dim_carriers (carrier_id)
 - fact_orders → dim_products (product_id)
 - bridge_products_per_plant → resolves warehouse-product mapping
 - analytics tables → derived from fact_orders
-
-
-

Entity Relationship Diagram



Database Schema Screenshot (RDS)



6. ETL Flow

Overview

The ETL (Extract, Transform, Load) process is a critical component of the Warehouse Network Carrier Performance Analytics Platform. It automates the movement of data from raw source files to the analytical data warehouse, ensuring that the data is accurate, consistent, and ready for business intelligence reporting.

The ETL pipeline is implemented using AWS services and PySpark to efficiently process large volumes of warehouse and carrier performance data.

Step 1: Extract

The extraction phase is responsible for collecting raw data from the source files and making it available for processing.

Activities Performed

- Read source CSV/Excel files.
- Upload datasets to the Amazon S3 Raw Bucket.
- Validate file availability and format.
- Trigger the ETL workflow automatically using AWS Lambda.

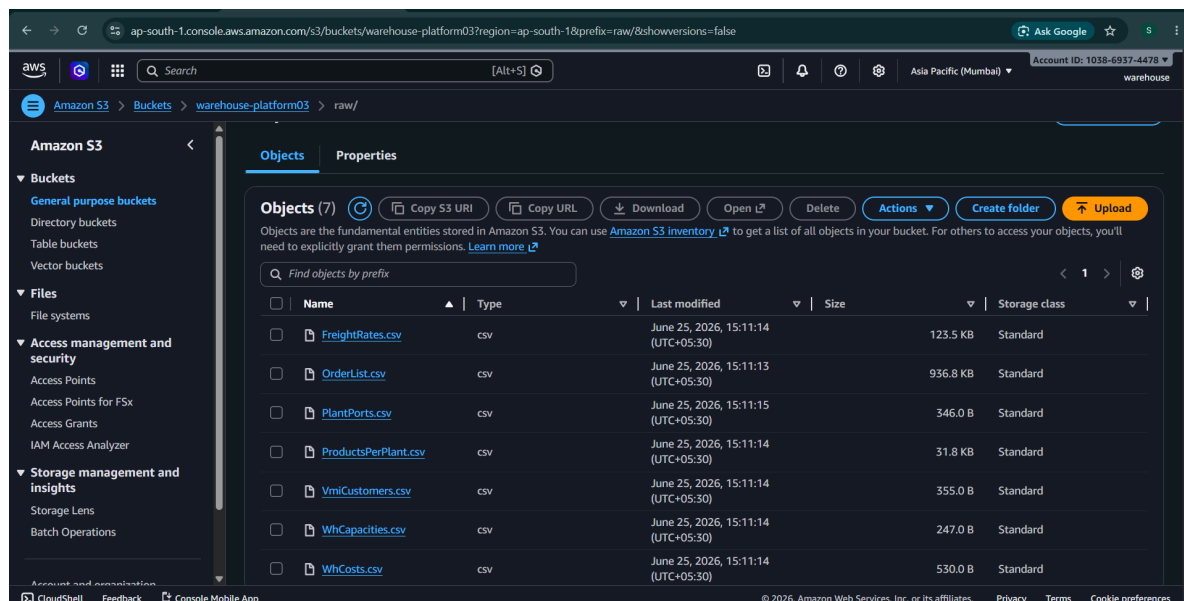
Input Files

The following datasets are extracted:

- OrderList
- FreightRates
- WhCapacities
- WhCosts
- ProductsPerPlant
- PlantPorts

Output

Raw datasets stored in Amazon S3 without modification.



Step 2: Transform

The transformation phase cleans, validates, and enriches the raw data using PySpark running on an Amazon EC2 instance.

Data Cleaning

The following data quality checks are performed:

- Removal of duplicate records.
- Handling missing or null values.
- Data type standardization.
- Column name normalization.
- Validation of mandatory fields.

Data Transformation

Business-specific transformations include:

- Joining warehouse and shipment datasets.
- Mapping products to warehouse locations.
- Combining carrier and freight rate information.
- Calculating shipment metrics.
- Preparing datasets for analytical reporting.

Data Validation

The transformed data is verified through:

- Row count validation.
- Schema validation.
- Data consistency checks.
- Relationship validation between datasets.

Output

Cleaned and enriched datasets ready for loading into the data warehouse.

Step 3: Load

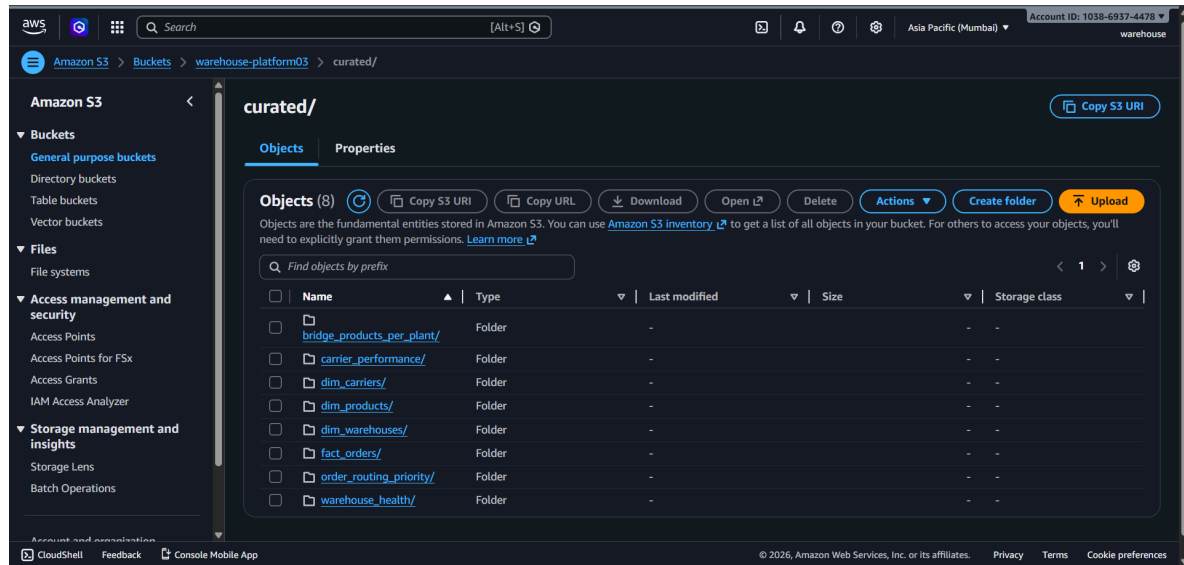
The load phase transfers the transformed data into the target analytics environment.

Activities Performed

- Store transformed datasets in the Amazon S3 Processed Bucket.
- Load processed data into RDS.
- Create analytical tables for reporting.
- Make data available for SQL querying and dashboard visualization.

Output

Structured and query-ready datasets stored in Amazon Redshift



Automation Workflow

The ETL pipeline is fully automated through event-driven processing.

Workflow Steps

1. User uploads a dataset to the S3 Raw Bucket.
2. S3 generates an event notification.
3. AWS Lambda is triggered automatically.
4. Lambda starts the ETL process on the EC2 instance.
5. PySpark performs extraction, transformation, and validation.
6. Processed data is written to the S3 Processed Bucket.
7. Data is loaded into Amazon Redshift.
8. SQL queries and dashboards consume the processed data.